

Building a CRUD API with SQLite3: A Case Study

By Princess Adekola

Voice cards: calm, plain, enthusiastic, no buzzwords, excited, and simple words

The Problem

This was an assignment from FlyRank AI, part of my backend AI Engineering track. The goal was simple: learn how backend systems actually work under the hood, from the ground up.

What I Did

I built a CRUD API in Python with a SQLite3 database. I covered all four operations: create, read, update, and delete. The endpoints included things like GET /tasks to get all tasks, and GET /tasks/3 to get a single one by ID. I could also update and delete tasks through the API.

For data validation, I used Pydantic. For handling errors properly, I used HTTPException so the right status codes came back instead of vague failures. I used DB Browser to look inside the database while I worked, and sqlite3 was imported directly into the project to handle the actual database connection.

It took me a week. I wasn't rushing, I wanted to understand each step before moving to the next.

What Came Of It

Everything worked well in the end. The only real hiccup was a missing comma, a small syntax slip, not a logic problem.

The biggest lesson was around security. I learned to use ? as a placeholder in my SQL queries and pass the actual values in as a tuple, instead of writing something like {task_id} directly into the query string. If you do it the direct way, someone could pass in something malicious disguised as a value, and it would get executed as a command. That's SQL injection, and now I understand why it matters and how to avoid it.

I also came away with a much better sense of why status codes matter, they're not just numbers, they tell the client exactly what happened.

This project is the foundation. Everything else I learn in backend AI engineering builds on this.

Princess Adekola Backend AI Engineering Track, FlyRank AI 
adekolaprincess02@gmail.com

Want to talk about this project or anything backend related? Email me.

AI Version (Before): Everything worked well in the end. The only real hiccup was a missing comma, a small syntax slip, not a logic problem.

My version (After): Everything worked well in the end. The only issue I had was missing a comma which is a syntax problem not logical.